Date:	Test - 1	Max. Marks:60
Semester:6th	UG	Duration: 2 Hrs
Course Title: Compiler Design		Course Code:CS363IA

Department of CSE

S No	Questions Part- A	M	BT	CO
1.	Define a lexeme. How it is related to a token?	2	1	1
2.	Write down the basic operations of a lexical analyzer.	2	3	3
3.	**Given the grammar below, left factor it:** D → while E do S | while E do S else S	1	2	2
4.	Left factor and remove any left recursion from the following: S→SS | S,S | a	2	1	1
5.	Draw two parse trees for the string id + id * id using the grammar: E → E + E | E * E | id	2	2	2
6.	Why can't LL(1) parsers handle ambiguous grammars?	1	1	1
	Part - B			

1.	Explain the major phases of a compiler and how they work together to translate high-level source code into target code. For each phase, identify a key challenge or design decision that affects overall compiler performance or correctness. Illustrate your answer with the given example showing how a simple code snippet is processed through each phase.
	int a,b;
	float c;
	a=9, b=6;
	c=a*b+5;
		10	2	2

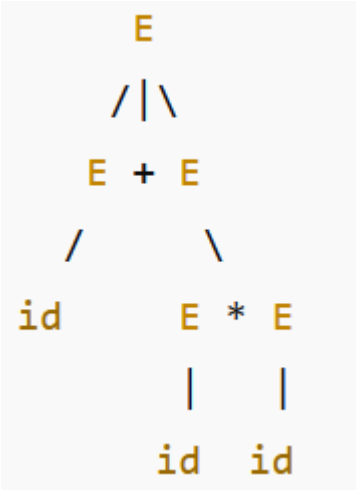
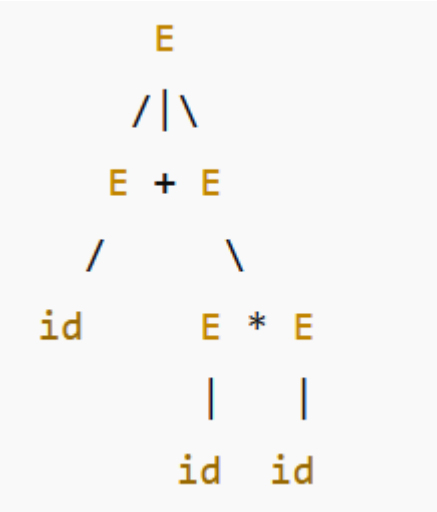
2.a	Explain how the twin-buffer (or double-buffer) scheme works in lexical analysis. Discuss its advantages over single-buffer approaches. Additionally, identify and explain two potential pitfalls or challenges in implementing the twin-buffer scheme and suggest how they can be resolved.	5	1	1
2.b	Lexical analyzers must distinguish between keywords, identifiers, operators, and literals efficiently. Explain how a lexical analyzer uses finite automata to recognize tokens. Why is the use of deterministic finite automata (DFA) preferred over nondeterministic finite automata (NFA) in implementation? Also, discuss how ambiguity is resolved when two token types share a common prefix.	5	3	3
3.	A grammar is said to be LL(1) if it can be parsed using a predictive parser with a single token of lookahead. Given the following grammar: $S \rightarrow Aa \mid bAc \mid dc \mid bda$ $A \rightarrow d$ **a)** Show whether the above grammar is LL(1) or not. Justify your answer with proper computation of FIRST and FOLLOW sets, and construction of the LL(1) parsing table. **(b)** If the grammar is not LL(1), explain **why** and suggest a transformation to make it LL(1)-compliant.	10	3	4
4.a	$S \rightarrow A\ B$ $A \rightarrow a\ A \mid \varepsilon$ $B \rightarrow b\ B \mid c$ (a) Explain how synchronization symbols are used in panic-mode error recovery in LL(1) parsers. (b) Identify the synchronization entries you would insert into the LL(1) parsing table for non-terminals A and B, and justify your choices.	5	4	3
4.b	Compute FIRST and FOLLOW set for the given grammar by making it suitable for LL(1) grammar. $S \rightarrow A\ a \mid b$ $A \rightarrow S\ c \mid d$	5	3	3

5.	S → stmt stmt → assign | ifstmt assign → id = expr ifstmt → if (cond) stmt else stmt expr → term expr' expr' → + term expr' | ε term → factor term' term' → * factor term' | ε factor → (expr) | id | num cond → expr relop expr relop → < | > | == Parse the string id=num+num*id using a predictive parsing table for the above grammar	10	4	4

Date:	**Test - 1**	**Max. Marks:60**
Semester:6th	**UG**	**Duration: 2 Hrs**
Course: Compiler Design		**Code:CS363IA**

SCHEME & SOLUTIONS

S No	Solutions with Scheme
1.	A **lexeme** is the **smallest meaningful unit** in the source code — essentially a sequence of characters that matches the pattern for a token. A **token** is a **category** or **class** (like IDENTIFIER, NUMBER, KEYWORD) that a lexeme belongs to. **Lexeme** = actual string in code (e.g., "while") **Token** = label/category (e.g., KEYWORD)
2.	1. Reading Input Characters 2. Tokenization 3. Removing Whitespace and Comments 4. Pattern Matching 5. Symbol Table Management 6. Error Handling
3.	After left factoring S□ SS’ | a S’□ S |,S After removing left recursion S□ aS” S” □ S’S” | ε S’ □ S | ,S

4.	E → T E' E' → + T E' | ε T → F T' T' → * F T' | ε F → id
5.	 
6.	Multiple Parse Trees = Confusion Parsing Table Conflicts

Part- B

1.	Solution: 1. Lexical Analysis (Scanner) • Function: Converts source code into tokens. • Output: Stream of tokens. • Example: For int a, b;, tokens: KEYWORD(int), IDENTIFIER(a), COMMA, IDENTIFIER(b), SEMICOLON • Key Challenge: Designing an efficient tokenizer that correctly handles errors and language keywords. 2. Syntax Analysis (Parser) • Function: Checks whether the token sequence follows the grammar of the language and constructs a parse tree or abstract syntax tree (AST).	**1** **1**

	• Example:
	For c = a * b + 5;, the AST will represent c as being assigned to the result of (a * b) + 5.
	
	• Key Challenge: Handling ambiguities and left recursion in the grammar.
	
	3. Semantic Analysis
	
	• Function: Ensures semantic correctness (e.g., type checking, scope resolution).
	
	• Example:
	Ensures c is a float and a * b + 5 results in a value that can be assigned to c.
	
	• Key Challenge: Detecting undeclared variables or incompatible types.
	
	4. Intermediate Code Generation
	
	• Function: Generates an intermediate representation (IR) that is machine-independent.
	
	• Example (Three-address code):
	
	5. Code Optimization
	
	• Function: Improves intermediate code to make it faster or consume less memory.
	
	• Example:
	Constant folding: If a = 9, b = 6, optimize a * b as 5*6
	
	5. Code Optimization
	
	• Function: Improves intermediate code to make it faster or consume less memory.
	
	• Example: t1 = 54
	
	• t2 = t1 + 5 → t2 = 59
	
	• c = t2
	Constant folding: If a = 9, b = 6, optimize a * b as 54
	
	6. Target Code Generation
	
	• Function: Converts IR into assembly or machine code.
	
	• Example (Assembly-like):
	
	MOV R1, 54
	
	ADD R1, 5
		1
		
		1.5
		
		1.5
		
		1

	MOV c, R1 7. Code Linking and Loading ● Function: Links compiled code with libraries and prepares it for execution. ● Key Challenge: Ensuring correct address resolution and handling external references. Diagram for Flow of all phases shown clearly.	**1** **2**
2.a	The **twin-buffer scheme** is a method used in the **lexical analyzer** to efficiently read and process characters from the source code. It uses **two buffers** of equal size to enable **seamless and fast input reading**. **How It Works:** 1. **Two Buffers:** The source file is read into two halves of a buffer alternately: Buffer 1 and Buffer 2. 2. **Forward Pointer:** A pointer (forward) scans the characters in the buffer to identify lexemes. 3. **Lexeme Begin Pointer:** Another pointer (lexeme_begin) marks the start of the current lexeme. 4. **Sentinel:** Each buffer ends with a **sentinel character** (usually EOF marker or null) to signal the buffer boundary. 5. **Switching Buffers:** o When forward crosses the sentinel at the end of Buffer 1, Buffer 2 is loaded from the source file. o Similarly, when Buffer 2 ends, Buffer 1 is refilled, and so on. 6. **No Backtracking Needed Across File:** Because buffers overlap logically, the analyzer can backtrack between them easily. **Advantages Over Single-Buffer Approach:** **Feature** **Single Buffer** **Twin Buffer** **I/O Efficiency** Frequent I/O calls Fewer I/O calls due to buffer swapping **Backtracking Support** Hard to implement across buffer end Easily done by managing two buffers with overlap	**2** **2** **1**

	Performance Slower due to I/O latency Faster and more efficient **Two Key Challenges & Solutions:** **1. Buffer Boundary Backtracking** • **Problem:** When a lexeme spans the end of one buffer and continues in the next, backtracking across the buffer boundary can be tricky. • **Solution:** Implement a **circular buffer** view or maintain logic to **move back** from Buffer 2 to Buffer 1 using pointer arithmetic. **2. Sentinel Confusion** • **Problem:** If the actual program contains a character identical to the sentinel (e.g., EOF-like character), it can be misinterpreted. • **Solution:** Use a **unique sentinel** not allowed in source code (e.g., a non-ASCII value) or distinguish real EOF using file stream metadata.	**1**
2.b	**1. Role of Finite Automata in Lexical Analysis** Lexical analyzers use **finite automata** to efficiently scan source code and identify valid **tokens** such as: • **Keywords** (e.g., int, while) • **Identifiers** (e.g., sum, x1) • **Literals** (e.g., 123, "hello") • **Operators** (e.g., +, ==, !=) A **regular expression** is written for each token type, and these are converted into **finite automata** for pattern matching. 🔹 **2. How It Works:** 1. **Each token type** is defined by a **regular expression**. 2. Regular expressions are **converted into NFAs** (nondeterministic finite automata). 3. NFAs are then **converted into DFAs** (deterministic finite automata) for actual implementation. 4. The lexical analyzer **runs the DFA** over the input stream and **accepts the longest matching prefix** as the token.	**1**

	🔹 **3. Why DFA is Preferred over NFA** **Feature NFA DFA** **Efficiency May require backtracking or multiple active states Single active state at any moment → faster** **Simplicity Complex to implement due to nondeterminism Easy table-driven implementation** **Determinism May need to explore multiple paths Always knows exactly what to do with each input symbol** **4. Resolving Ambiguity (Longest Match and Priority Rules)** **When two token types share a common prefix (e.g., int can be both a keyword and an identifier):** **Rules Used:** 1. **Longest Match Rule: The lexer reads the longest possible string that forms a valid token.** o **Example: For <=, it should recognize <= as a single token, not < and =.** 2. **Rule Priority: If multiple token types match the same string, the lexer assigns the token type with higher priority (usually keywords are ranked above identifiers).**	**1.5** **1.5**
3.	**a. Step 1: Compute FIRST Sets** **FIRST(S) = { b, d }** **Step 2: Compute FOLLOW set** **FOLLOW(S) = { $ }** **FOLLOW(A) = { a, c }** **Step 3: Build LL (1) Parsing Table** **So we have conflicts in parsing table entries for S:** • **Two rules for S on input d → conflict between Aa and dc**	**1** **2** **2** **1**

	• **Two rules for S on input b → conflict between bAc and bda** **(b) Why Not LL(1), and How to Transform** **There are FIRST/FIRST conflicts in the rules of S, specifically:** • **Aa and dc both start with d** • **bAc and bda both start with b** **Solution: Apply Left Factoring:** **Let's factor out common prefixes in S:** **S → b S' | d S''** **S' → A c | d a** **S'' → a | c** **A → d**	**2** **2**
4.a	**Step 1: FIRST and FOLLOW Sets** **FIRST(A) = { a, ε } FIRST(B) = { b, c } FIRST(S) = { a, b, c }** **Step 2: FOLLOW set** **FOLLOW(S) = { $ } FOLLOW(A) = FIRST(B) = { b, c }** **FOLLOW(B) = FOLLOW(S) = { $ }** **Step 3: LL(1) Parsing table** **Non-terminal Terminal Production** **A a A → a A** **A b, c A → ε** **B b B → b B** **B c B → c** **Step 4: Synchronization input**	**1** **1** **1** **2**
4.b	**Step 1: Eliminate Left Recursion** **We see S calls A, and A calls back S, making indirect left recursion.**	**2**

	Let's substitute A → S c into S → A a: **Substitute A into S: S → (S c) a | d a | b** **S → S c a | d a | b** **Removing left recursion: S → d a S' | b S'** **S' → c a S' | ε** **Step 2: Compute FIRST and FOLLOW Sets** **FIRST(d a S') = { d }** **FIRST(b S') = { b }⇒ FIRST(S) = { d, b }** **FIRST(c a S') = { c }, FIRST(ε) = { ε }⇒ FIRST(S') = { c, ε }** **FOLLOW Sets: FOLLOW(S) = { $ } FOLLOW(S') = FOLLOW(S) = { $ }** **FOLLOW(S) = { $ } FOLLOW(S') = { $ }**	**1** **2**
5.	**Predictive Parsing Table (Partial for Relevant Input)** **Parsing Steps:** **Parsing Steps:** **Stack — Input — Action / Rule Used** **S $ — id = num + num * id $ — S → stmt** **stmt $ — id = num + num * id $ — stmt → assign** **assign $ — id = num + num * id $ — assign → id = expr** **id = expr $ — id = num + num * id $ — Match id** **= expr $ — = num + num * id $ — Match =** **expr $ — num + num * id $ — expr → term expr'** **term expr' $ — num + num * id $ — term → factor term'** **factor term' expr' $ — num + num * id $ — factor → num** **term' expr' $ — + num * id $ — term' → ε** **expr' $ — + num * id $ — expr' → + term expr'** **+ term expr' $ — + num * id $ — Match +**	**10**

	term expr' $	**num * id $**	**term → factor term'**	
	factor term' expr' $	**num * id $**	**factor → num**	
	term' expr' $	*** id $**	**term' → * factor term'**	
	*** factor term' expr' $**	*** id $**	**Match ***	
	factor term' expr' $	**id $**	**factor → id**	
	term' expr' $	**$**	**term' → ε**	
	expr' $	**$**	**expr' → ε**	
	$	**$**	**ACCEPT**	